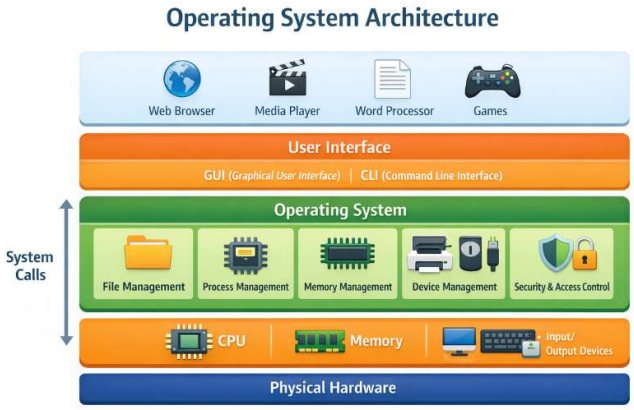
LINUX COMMANDS-CLASS

Q.What is an Operating System (OS)?

An Operating System (OS) is system software that acts as an interface between the user, application programs, and computer hardware.
It controls and coordinates the use of hardware resources and provides services for running programs.

In simple terms:
The OS makes the computer usable by managing hardware and allowing users and applications to interact with it.



Functionality of Operating System (OS) — Short with Example

1. **Process Management**
   Controls running programs and CPU time.
   Example: Running a browser and music player at the same time.

2. **Memory Management**
   Allocates RAM to programs.
   Example: Giving 2 GB RAM to a game and 1 GB to a text editor.

3. **File Management**
   Organizes and manages files and folders.
   Example: Saving a document in the "Documents" folder.

4. **Device Management**
   Controls hardware devices.
   Example: Making a printer print a file.

5. **User Interface**
   Provides interaction method.
   Example: Clicking icons in Windows or typing commands in Linux.

6. **Security Management**
   Protects system and data.
   Example: Asking for a password to log in.

7. **Resource Management**
   Shares hardware fairly.
   Example: Giving more CPU to a video call than a background download.

8. **Error Handling**
   Finds and fixes errors.
   Example: Showing "Disk error" message.

9. **Networking**
   Manages network communication.
   Example: Sending an email using internet.

LINUX HISTORY:

1969 – UNIX

- **UNIX was developed at Bell Labs by Ken Thompson and Dennis Ritchie.**
- **It became the foundation for many modern operating systems.**

1983 – Richard Stallman

- **Richard Stallman started the GNU Project.**
- **Goal: create a free Unix-like operating system.**
- **GNU created many tools but did not have a working kernel.**

1991 – Linus Torvalds

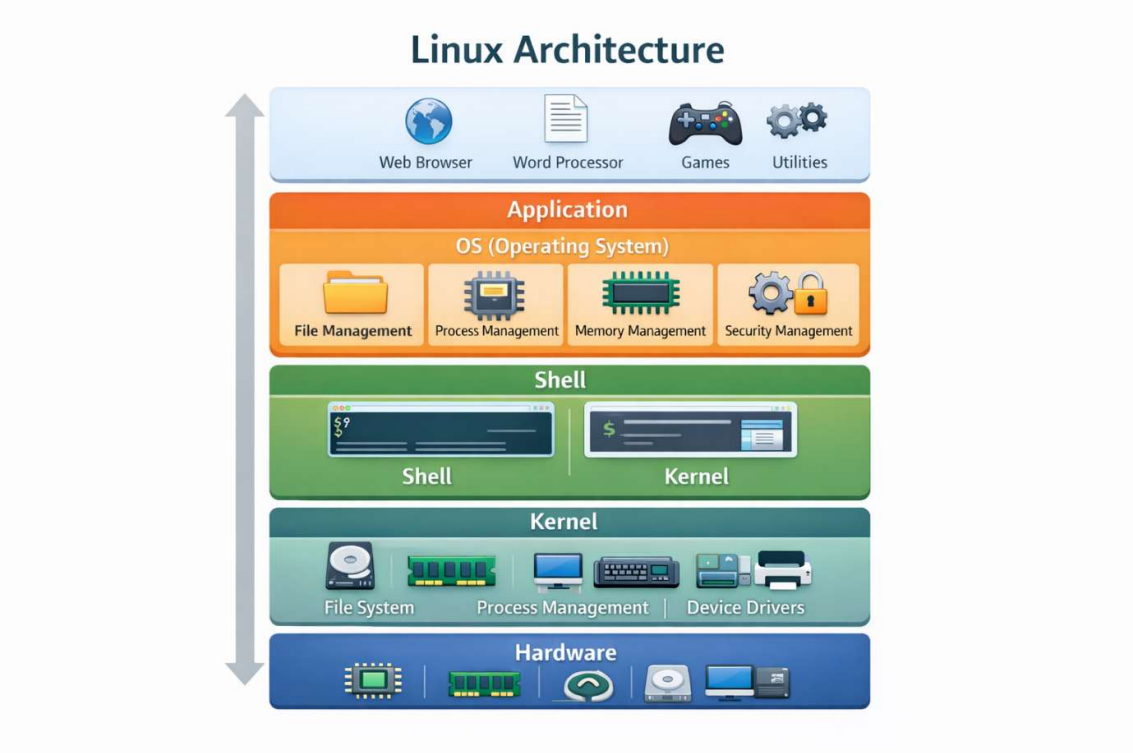
- **Linus Torvalds developed a kernel as a student project.**
- **He first named it "Freax" (Free + Freak + Unix).**
- **He announced it on August 25, 1991.**

1992 – Linux

- **The kernel was renamed "Linux."**
- **Linux was released under the GNU General Public License (GPL).**
- **Linux + GNU tools formed a complete operating system (GNU/Linux).**

Q.what is linux?

Linux is a free and open-source, Unix-like operating system kernel that manages hardware resources and provides services for applications, forming the core of many operating systems such as Ubuntu, Fedora, and Debian.



Properties of Linux:

1. Free and Open Source
   Linux is released under the GNU General Public License (GPL).
   Its source code is freely available to view, modify, and redistribute.

2. Filesystem Hierarchy
   Linux follows a standard directory structure called the Filesystem Hierarchy Standard (FHS).
   Examples:
   / – root directory
   /home – user files
   /etc – configuration files
   /bin, /usr/bin – essential programs
   /var – logs and variable data

3. Multitasking
   Linux can run many programs at the same time.
   The kernel schedules processes so multiple tasks share CPU time efficiently.

4. Multiuser Management
   Many users can log in and work at the same time.
   Each user has separate permissions, files, and processes.

5. Memory Management
   Linux manages RAM using:
   – Virtual memory
   – Paging and swapping

– Efficient allocation and deallocation
This allows large programs to run even if physical RAM is limited.

6. Security Management
   Linux has strong security features:
   – User and group permissions
   – File access control (read, write, execute)
   – Security modules like SELinux and AppArmor
   – Regular security updates

7. Easily Customizable
   Linux can be changed at almost every level:
   – Desktop environment
   – Kernel configuration
   – Installed software
   – System services
   Users can build a system that fits their exact needs.

Conclusion:
Free and open source, filesystem hierarchy, multitasking, multiuser management, memory management, security management, and easy customization are all fundamental properties of Linux. Linux commands are used to control, configure, and take advantage of these properties, but the properties themselves belong to the Linux operating system

LINUX_COMMANDS:

1. ls

Lists all files and directories in the current location.
Example:
ls

---

2. mkdir

Creates a new directory (folder).
Example:
mkdir devops

---

3. ls -l

Displays detailed information about files and directories, including permissions, owner, size, and timestamps.
Example:
ls -l

---

4. pwd

Shows the current working directory.
Example:
pwd
Output example: /home/ubuntu

---

5. touch

Creates a new empty file or updates the timestamp of an existing file.
Example:
touch newfile.txt
Note: In many terminals, files appear in white and directories in blue when using ls.

---

6. cd

Changes the current directory.
Example:
cd devops

---

7. cd ..

Moves to the previous (parent) directory.
Example:
cd ..

---

8. rm

Deletes a file.
Example:
rm devops_file.txt

---

9. rm -r

Deletes a directory and all files inside it (recursive delete).
Example:
rm -r cloud

---

10. rmdir

Deletes an empty directory.
Example:
rmdir devops

---

11. echo

Prints text to the terminal.
Example:
echo "hello dosto"

---

12. echo with redirection (>)

Writes content into a file.
If the file does not exist, it gets created.
Example:
echo "hello dosto" > demofile.txt

---

13. cat

Displays the content of a file.
Example:
cat demofile.txt
Output: hello dosto

Another example:
echo "this is my file" > myfile.txt
(This creates the file and writes into it.)

---

14. zcat

Displays the content of a compressed .gz file without unzipping it.
Example:
zcat logfile.gz

---

15. head

Shows the first few lines of a file (default is 10 lines).
Example:
head myfile.txt

---

16. tail

Shows the last few lines of a file.
Example:
tail myfile.txt

---

17. tail -f

Displays new lines added to a file in real time (useful for logs).
Example:
tail -f filename.txt

---

18. less

Views large files page by page.
Example:
less myfile.txt

---

19. more

Similar to less but with fewer navigation options.
Example:
more myfile.txt

---

20. cp

Copies a file from one location to another.
Example:
cp newfile.txt devops/

---

21. Copying a file from one folder to another

Example:
cp demofile.txt(source) linux_for_devops(destination)

---

22. cp -r

Copies an entire folder recursively.
Example:
cp -r cloud(source) linux_for_devops/(destination)

---

23. mv

Moves files or directories from one location to another.
Example:
Move a file from devops to cloud:
mv newfile.txt cloud

---

24. Renaming a directory

Uses mv.
Example:
mv devops linux_for_devops

---

25. wc

Counts the number of lines, words, and bytes in a file.
Example:
wc myfile.txt

1. mkdir Command

mkdir -p India/karnataka/gadag

- Creates directories in hierarchy.
- -p means “create parent folders if they do not exist.”

Hierarchy created:

India
└── karnataka
    └── gadag

---

2. cat Command

cat > filename

- Creates a file and allows you to write data.
- Overwrites existing data if file already exists.
- Press Ctrl + D to save and exit.

cat >> filename

- Appends data to an existing file.
- Old data is not deleted.

---

3. nano Editor

nano filename.txt

- Opens file in nano editor.
- Ctrl + S → Save file
- Ctrl + X → Exit editor

---

4. Path

A path means the location of a file or directory.

There are two types:

1) Absolute Path

- Full path starting from root /.

Example:

touch /home/fortunecloud/dir1/gadag/a.txt

This means:
/ → home → fortunecloud → dir1 → gadag → a.txt

2) Relative Path

- Path from current directory.

Symbols:
./ → current directory
../ → previous directory

Examples:

touch ./test/b.txt
Creates file inside “test” folder in current directory.

touch ../data/c.txt
Creates file inside “data” folder in previous directory.

Extra: Tree Structure Example

If you run:

mkdir -p school/class1/sectionA

Structure:

```
school
└── class1
    └── sectionA
```

Summary:

- mkdir -p → create folders in hierarchy
- cat > file → write new data
- cat >> file → append data
- nano → text editor
- Absolute path → full path from /

- Relative path → path from current location.

#USER LEVEL COMMANDS(fortune):

1. cat /etc/passwd

Command:
cat /etc/passwd

Explanation:

- **Displays configuration details of all users.**
- **Each line represents one user.**
- **Shows: username, UID, GID, home directory, shell.**

Example output line:
john:x:1001:1001:John:/home/john:/bin/bash

---

2. getent passwd

Command:
getent passwd

Explanation:

- **Shows user details from system databases.**
- **Includes local users and network users (LDAP, etc.).**
- **Similar to /etc/passwd but more complete.**

---

3. adduser and useradd

adduser username

Explanation:

- **Interactive command.**
- **Asks for:**
  - **Password**
  - **UID, GID**
  - **Default home directory**
  - **User information**
- **Default shell usually /bin/bash.**

Example:
adduser ravi

---

useradd username

Explanation:

- **Non-interactive.**
- **Creates user with default settings.**
- **Default shell usually /bin/sh.**
- **Does not ask for password automatically.**

Example:
useradd ram
passwd ram

---

4. su (Switch User)

Command:
su

Explanation:

- **Switches to root user by default.**
- **Asks for root password.**

Example:
su

---

5. su -

Command:
su -

Explanation:

- **Switches user and loads full login environment.**
- **Changes home directory, shell settings, variables.**

Example:
su - root

---

6. passwd username

Command:
passwd john

Explanation:

- **Changes password of a user.**

---

7. Delete User

userdel username

Explanation:

- **Deletes only user account.**
- **Home directory is not deleted.**

userdel -r username

Explanation:

- **Deletes user and home directory.**

---

8. Rename User

usermod -l newname oldname

Explanation:

- **Changes username.**

Example:
usermod -l ravi ram

---

9. Change Home Directory

usermod -d /new/home -m username

Explanation:

- **Changes home directory.**
- **-m moves old data to new directory.**

Example:
usermod -d /home/newuser -m ravi

---

10. Other Commands

ls /home

Explanation:

- **Lists all user home directories.**

echo $0

Explanation:

- **Shows current shell name.**

Example output:
bash

Group Management in Linux

---

1. cat /etc/group

Command:
cat /etc/group

Explanation:

- Displays all group details.
- Each line shows: group name, password field, GID, group members.

Example line:
dev:x:1002:john,ram

---

2. getent group

Command:
getent group

Explanation:

- Shows group information from system databases.
- Includes local and network groups.

---

3. Create Group

Command:
groupadd grpname

Explanation:

- Creates a new group.

Example:
groupadd developers

---

4. Add User to Group

Command:
usermod -aG groupname username

Explanation:

- Adds user to supplementary group.
- -a = append, -G = group list.

Example:
usermod -aG developers john

---

5. Remove User from Group

Command:
gpasswd -d username groupname

Explanation:

- Deletes user from a group.

Example:
gpasswd -d john developers

---

6. Delete Group

Command:
groupdel groupname

Explanation:

- Removes a group from system.

Example:
groupdel developers

Types of Groups

Linux groups are of two types:

1. Primary Group

- Each user has exactly one primary group.
- Files created by user belong to this group by default.
- Stored in /etc/passwd.

Example:
User john primary group = john

---

2. Supplementary (Associated) Group

- User can belong to many extra groups.

- Used to give additional permissions.
- Stored in /etc/group.

Example:
User john is in: developers, admin, testing

1. Root User

- Root is the super administrator.
- Has no restrictions.
- Can perform any operation.
- User ID (UID) = 0

Example:
whoami → root

---

2. System Users

- Created automatically during system installation.
- Used by services and system processes.
- UID range: 1 – 999
- Examples: daemon, bin, sys, lp

Purpose:

- Run background services securely.
- Not used for login normally.

---

3. Normal Users

- Created after system installation.
- Used by real users.
- UID range: 1000 – 5999 (may vary by distro).
- First normal user usually has UID 1000.

Features:

- Limited permissions.
- Cannot change system files directly.
- Can become root using sudo if allowed.

Example:
sudo yum install httpd

---

4. Super User (Sudo User)

- A normal user who is added to sudo group.
- Can run admin commands using sudo.
- Acts like root temporarily.

Example:
Add user to sudo group:
usermod -aG sudo john (Debian/Ubuntu)
usermod -aG wheel john (RHEL/CentOS)

Use:
sudo reboot.

Scenario

Assume:

- Your home directory is:
  /home/ubuntu
- Inside it, you have two folders:
  - demo → contains a.txt
  - test → contains b.txt

So the structure is:

/home/ubuntu/

├── demo/

│ └── a.txt

└── test/

└── b.txt

Copy (cp) Command

Using Absolute Path

Absolute path means you write the full path from root (/).

cp /home/ubuntu/demo/a.txt /home/ubuntu/test/

This copies a.txt from demo to test.

Using Relative Path

Relative path is based on your current directory.
If you are inside /home/ubuntu, then:

cp ./demo/a.txt ./test/

./ means “current directory”.

Move (mv) Command

Move a File Using Absolute Path

mv /home/ubuntu/test/b.txt /home/ubuntu/demo/

This moves b.txt from test to demo.

Rename a File Using mv

Renaming is also done using mv.

Using Absolute Path

mv /home/ubuntu/test/a.txt /home/ubuntu/test/c.txt

This renames a.txt to c.txt in the same folder.

Using Relative Path

If you are inside /home/ubuntu/demo and want to rename a file in test:

mv ./test/a.txt ../test/c.txt

.. means “one directory up”.

Path (Human Readable Path)

A path is the location of a file or folder in the system, for example:

- Absolute path: /home/ubuntu/demo/a.txt
- Relative path: ./demo/a.txt

.bash_history

.bash_history is a hidden file in your home directory.

It stores all the commands you have typed in the terminal.

You can view it using:

cat ~/.bash_history

or

less ~/.bash_history.

File Permissions – Clean Conceptual Explanation

1. What is a File?

A **file** is a resource where data is stored (text, script, program, etc.).

2. What is Permission?

A **permission** defines what actions a user can perform on a file or directory.

Types of Permissions

Permission	Meaning	Symbol	Numeric
Read	View file content	r	4
Write	Edit/add/delete content	w	2
Execute	Run a program/script	x	1

Combined Values

Number	Meaning
7	rwx (read + write + execute)
6	rw- (read + write)
5	r-x (read + execute)
4	r-- (read only)
3	-wx (write + execute)
2	-w- (write only)
1	--x (execute only)
0	--- (no permission)

Types of Users

User Type	Meaning	Symbol
Owner/User	Person who created file	u
Group	Group of users	g
Others	Everyone else	o

Example: Permission 400

4 0 0

u g o

r-- --- ---

- Owner: read only
- Group: no permission
- Others: no permission

Viewing Permissions: ls -l

Example:

drwxr-xr-x 2 vinayak vinayak 4096 Jan 14 12:22 demo

Breakdown:

Part	Meaning
d	Directory (- means file)
rwx	Owner permissions
r-x	Group permissions
r-x	Others permissions
2	Number of links
vinayak	Owner
vinayak	Group
4096	Size in bytes
Jan 14 12:22	Date/time
demo	File/Folder name

drwxr-xr-x Explained

d rwx r-x r-x

| | | |

| | | +--> Others: read & execute

| | +--------> Group: read & execute

| +----------> Owner: read, write, execute

+----------> Directory

File Listing Example

-rw-r--r-- 1 vinayak vinayak 6 Jan 17 17:00 a.txt

Meaning:

- rw- r-- r--

| | | |

| | | +--> Others: read

| | +--------> Group: read

| +----------> Owner: read & write

+----------> File

Changing Permissions: chmod

Symbolic Method

- Add permission:
- chmod u+x sample.sh
- Remove permission:
- chmod u-x sample.sh
- Overwrite permission:
- chmod u=rwx sample.sh

Numeric Method

chmod 755 file

Means:

7 → owner: rwx

5 → group: r-x

5 → others: r-x

Your Example

Before:

-rw-r--r-- sample.sh

After:

chmod u+x sample.sh

Now:

-rwxr--r-- sample.sh

Meaning:

- Owner can execute now

Changing Owner and Group

Change Owner

chown darshan sample.sh

Change Owner Recursively

chown -r darshan demo

Change Group

chgrp powerstar sample.sh

Change Both

chown darshan:powerstar sample.sh

Final Output You Showed

-rwxr-xr-- 1 darshan powerstar 19 Jan 17 17:01 sample.sh

Breakdown:

- rwx r-x r--

| | | |

| | | +--> Others: read only

| | +--------> Group: read & execute

| +--------------> Owner: read, write, execute

+------------------> File

Owner = darshan
Group = powerstar

Group Commands You Used

sudo groupadd powerstar

sudo usermod -aG powerstar darshan

getent group powerstar

Meaning:

- Created group
- Added user to group
- Verified group.